

Formalizing the homotopy extension property in LEAN

Mara Masike Silge

Born on 24. April 2003 in Werne

August 17, 2026

Bachelor's Thesis Mathematics

Advisor: Prof. Dr. Floris van Doorn

Second Advisor: Prof. Dr. Markus Hausmann

MATHEMATISCHES INSTITUT

MATHEMATISCH-NATURWISSENSCHAFTLICHE FAKULTÄT DER
RHEINISCHEN FRIEDRICH-WILHELMS-UNIVERSITÄT BONN

Zusammenfassung

Motivation: Whitehead theorem oder so?

Contents

1	Vorbemerkung	3
1.1	Motivation	3
2	Homotopy Extension Property	4
2.1	The Homotopy Extension Property	4
2.2	HEP for relative CW-complexes	8
2.2.1	HEP for balls	9
2.2.2	HEP for consecutive skeletons	10
2.2.3	HEP for relative CW complexes	13
3	Lean and mathlib	15
3.1	Lean's Dependent Type Theory	15
3.2	Sets and Subtypes	16
3.3	Structures	16
3.4	Junk Values	17
3.5	mathlib	17
4	Lean formalisation of the Homotopy Extension Property	19
4.1	CW-complexes in Mathlib	19
4.2	HEP Definition	19
4.3	Retraction Criterion	22
4.4	Preservation of the HEP	24
4.5	HEP for discs and cubes	25
4.6	HEP for consecutive skeletons	27
4.6.1	Skeleton is a quotient space	27
4.6.2	construction of the retraction	29
4.6.3	Continuity of the retraction	31
4.6.4	Properties of the retraction	32
4.7	HEP for finite dimensional relative CW complexes	33
5	Methodes and Usage of AI	35
6	Summary und Ausblick	36
	Literatur	37

1 Vorbemerkung

In this work, if not specified in any other way, every space has a topology and every map is a continuous map.

1.1 Motivation

The goal of this thesis is to formalize the *homotopy extension property (HEP)* in Lean. I started with the proof of the *retraction criterion* and worked towards the result that every relative CW complex satisfies the HEP. Our motivation for studying this property is that it is needed for two major theorems in algebraic topology. The first is the *cellular approximation theorem*.

Theorem 1.1 (Cellular Approximation Theorem, [MR1867354]). *Every map $f: X \rightarrow Y$ of CW complexes is homotopic to a cellular map. If f is already cellular on a subcomplex $A \subset X$, the homotopy may be taken to be stationary on A .*

Here a map is called *cellular* if it sends the n -skeleton of X to the n -skeleton of Y for all $n \in \mathbb{N}$.

Equipping S^k with the CW structure consisting of one 0-cell and one k -cell, it has no cells in dimensions $0 < n < k$. Hence any map $S^n \rightarrow S^k$ with $n < k$ is homotopic to one landing in the n -skeleton, which is a single point, and is therefore nullhomotopic. This yields the corollary $\pi_n(S^k) = 0$ for all $n < k$.

Secondly, the result that relative CW complexes satisfy the HEP is one of the main steps towards the proof of *Whitehead's theorem*.

Theorem 1.2 (Whitehead's Theorem, [MR1867354]). *If a map $f: X \rightarrow Y$ between connected CW complexes induces isomorphisms $f_*: \pi_n(X) \rightarrow \pi_n(Y)$ for all n , then f is a homotopy equivalence. In case f is the inclusion of a subcomplex $X \hookrightarrow Y$, the conclusion is stronger: X is a deformation retract of Y .*

Whitehead's theorem states that, in order to prove that two connected CW complexes are homotopy equivalent, it suffices to exhibit a map between them that induces isomorphisms on all homotopy groups, i.e. a weak homotopy equivalence. Showing that two spaces are homotopy equivalent is in general a difficult task, and Whitehead's theorem provides a powerful tool for such problems.

2 Homotopy Extension Property

Our formalisation is not based on a single source. The overall structure follows the lecture notes of the course Topology I, taught by Markus Hausmann at the University of Bonn in the winter term 2025/26. These notes are not publicly available; the chapter on the homotopy extension property closely follows the (German) lecture notes Algebraische Topologie by F. Waldhausen (Bielefeld) [1], which we cite instead wherever a specific argument is taken from them. Throughout, all maps between topological spaces are assumed to be continuous, and we write $I := [0, 1]$ for the unit interval.

2.1 The Homotopy Extension Property

There are several equivalent ways to define the homotopy extension property, and this section We begin with the definition that is closest to the one we formalised in Lean.

– Question: nicht genau so zitiert, aber es gibt ähnliche definition in der Quelle, dann schribbe ich die Quelle dran?

Definition 2.1. ?? A pair of topological spaces (X, A) with $A \subseteq X$ has the *homotopy extension property (HEP)* if for every topological space Y , every map $f: X \rightarrow Y$, and every homotopy $H: A \times I \rightarrow Y$ satisfying $H(a, 0) = f(a)$ for all $a \in A$, there exists a homotopy $H': X \times [0, 1] \rightarrow Y$ such that

$$\begin{aligned} H'(x, 0) &= f(x) && \text{for all } x \in X, \\ H'(a, t) &= H(a, t) && \text{for all } (a, t) \in A \times [0, 1]. \end{aligned}$$

The two extreme cases are immediate, but we record them separately because we will use them later as the base case of an induction.

Lemma 2.2. *For every topological space X , both (X, X) and $(X, \emptyset)$ have the HEP.*

Proof. For (X, X) , the homotopy H is already defined on all of $X \times I$ and satisfies $H(x, 0) = f(x)$, so we may take $H' := H$.

For $(X, \emptyset)$, the second condition of theorem 2.1 is vacuous, so the constant homotopy $H'(x, t) := f(x)$ has the required properties. $\square$

Remark 2.3. The HEP can equivalently be phrased as follows: (X, A) has the *HEP* if and only if every map defined on the pushout $X \cup_A (A \times I)$ extends to a map on $X \times I$. Here the pushout is taken along the inclusion $A \hookrightarrow X$ and the map $A \rightarrow A \times I, a \mapsto (a, 0)$. This is in general, *not* the same as requiring that every map defined on the subspace $X \times \{0\} \cup (A \times I) \subset X \times I$ extends to $X \times I$, because the canonical map

$$X \cup_A (A \times I) \rightarrow X \times \{0\} \cup (A \times I)$$

need not be a necessarily a homeomorphism. In general, it is only a continuous bijection. If A is closed in X , however, both $X \times \{0\}$ and $A \times I$ are closed in $X \times I$, the canonical map is a closed map and hence a homeomorphism, and the two formulations agree. This is the reason why the closedness assumption appears in exactly one direction of theorem 2.4 below.

Lemma 2.4. *Let A be a subset of a topological space X and consider the following two statements*

(i) *The pair (X, A) has the HEP*

(ii) *For every topological space Y and every continuous map $g: (X \times \{0\}) \cup (A \times I) \rightarrow Y$ there exists an extension of g to a map $G: X \times I \rightarrow Y$*

Then (i) implies (ii). If in addition A is closed in X , then also (ii) implies (i).

Proof. (i) $\Rightarrow$ (ii). Let $g: X \times \{0\} \cup A \times I \rightarrow Y$ be a map. Define

$$f: X \rightarrow Y, x \mapsto g(x, 0) \quad \text{and} \quad H: A \times [0, 1] \rightarrow Y, (x, t) \mapsto g(x, t).$$

Both are continuous, being composites of g with the continuous maps $x \mapsto (x, 0)$ and $A \times I \hookrightarrow (X \times \{0\}) \cup (A \times I)$. They are compatible in the sense of theorem 2.1, since $H(a, 0) = g(a, 0) = f(a)$ for all $a \in A$. Applying the HEP of (X, A) yields a homotopy $H': X \times I \rightarrow Y$ with $H'(x, 0) = f(x) = g(x, 0)$ and $H'|_{A \times I} = H = g|_{A \times I}$. Hence $G := H'$ restricts to g on all of $(X \times \{0\}) \cup (A \times I)$.

(ii) $\Rightarrow$ (i), assuming A closed. Let $f: X \rightarrow Y$ and $H: A \times I \rightarrow Y$ with $H(a, 0) = f(a)$ for all $a \in A$. We define

$$g: X \times \{0\} \cup A \times [0, 1] \rightarrow Y, \quad (x, t) \mapsto \begin{cases} f(x), & t = 0, \\ H(x, t), & \text{otherwise} \end{cases}$$

The map g is well defined: the two cases overlap precisely on $A \times \{0\}$, where $H(a, 0) = f(a)$ by assumption. For continuity we use that A is closed in X : then $A \times I$ is closed in $X \times I$, and so is $X \times \{0\}$. Both are therefore closed in the subspace $(X \times \{0\}) \cup (A \times I)$, which they cover, and the restrictions of g to them are continuous. By assumption there is an extension $G: X \times I$ of g , and $H' := G$ satisfies both conditions of theorem 2.1 by construction. $\square$

Definition 2.5. Let X be a topological space and $A \subset X$ a subset with inclusion $\iota: A \hookrightarrow X$. A *retraction* of X onto A is a map $r: X \rightarrow A$ with $r \circ \iota = id_A$. If such an r exists, we call A a *retract* of X .

Lemma 2.6. *Let A be a subset of a topological space X . The following are equivalent:*

(i) For every topological space Y and every $g: A \rightarrow Y$ there exists an extension $G: X \rightarrow Y$

(ii) A is a retract of X .

Proof. (i) $\Rightarrow$ (ii). Apply (i) to the special case $Y := A$ and $g := id_A$. The resulting extension G restricts to the identity on A and is therefore a retraction.

(ii) $\Rightarrow$ (i). Let $g: A \rightarrow Y$ be a map and let $r: X \rightarrow A$ be a retraction. Then $G := g \circ r$ is continuous, and for $a \in A$ we have $G(a) = g(r(a)) = g(a)$, so G extends g . $\square$

Combining the two lemmas gives the *retraction criterion*, which is the tool we use in several of the following proofs: it replaces the quantification over all spaces Y and all maps into them by the existence of a single map. In the lecture notes the criterion is stated only for closed subspaces $A \subseteq X$. Hatcher [Hat02] states it without any such assumption, but his proof of the implication from the retraction to the HEP again requires A to be closed; for the general case he refers to the appendix, where a detailed proof following an argument of [Str68] is given. Our formulation follows the same pattern as theorem 2.4: we impose closedness only on the implication that actually needs it.

For our purposes this restriction is harmless, since in all cases we consider the closedness of the subspace is available at no additional cost. –ToDo : Ausblick: verallgemeinern

Proposition 2.7 (Retraction Criterion). *Let X be a topological space and $A \subseteq X$ a subspace. Consider the following two statements:*

(i) The pair (X, A) has the HEP

(ii) the subspace $X \times \{0\} \cup A \times I$ is a retract of $X \times I$

Then (i) implies (ii). If in addition A is closed in X , then (ii) also implies (i).

Proof. (i) $\Rightarrow$ (ii). By theorem 2.4, every map defined on $(X \times \{0\}) \cup (A \times I)$ extends to $X \times I$. Applying theorem 2.6 to the subspace $(X \times \{0\}) \cup (A \times I) \subseteq X \times I$ yields the retraction.

(ii) $\Rightarrow$ (i), assuming A closed. By theorem 2.6, applied to the same subspace, every map defined on $(X \times \{0\}) \cup (A \times I)$ extends to $X \times I$. Since A is closed in X , the second direction of theorem 2.4 applies and yields the HEP for (X, A) . $\square$

The remaining results of this section are permanence properties: they allow us to build new pairs with the HEP out of known ones. The first one is used in ?? to transfer the HEP from balls to cubes, the second one is the inductive step in the proof for relative CW-complexes.

Proposition 2.8. *Let (X, A) and (Y, B) be pairs of topological spaces with B closed in Y , and let $\varphi: X \rightarrow Y$ be a homeomorphism with $\varphi(A) = B$. If (X, A) has the HEP, then so does (Y, B) .*

Proof. Applying theorem 2.7 to (X, A) gives a retraction $r_X: X \times I \rightarrow (X \times \{0\}) \cup (A \times I)$. Let $pr_X: X \times I \rightarrow X$ and $pr_I: X \times I \rightarrow I$ denote the projections onto the first and second component, respectively, and define

$$r_Y: Y \times I \longrightarrow (Y \times \{0\}) \cup (B \times I)$$

$$(y, t) \longmapsto \left(\varphi \circ pr_X \circ r_X(\varphi^{-1}(y), t), pr_I \circ r_X(\varphi^{-1}(y), t) \right).$$

We show that r_Y is a retraction. By theorem 2.7 this yields the HEP for the pair (Y, B) , since B is closed in Y . The map r_Y

- *is continuous*, since both component functions are composites of continuous maps.
- *takes values in $(Y \times \{0\}) \cup (B \times I)$* : let $(y, t) \in Y \times I$. The point $r_X(\varphi^{-1}(y), t)$ lies in the image of r_X , which is $(X \times \{0\}) \cup (A \times I)$. Hence either its second component is 0, in which case the second component of $r_Y(y, t)$ is 0 as well, or its first component lies in A , in which case the first component of $r_Y(y, t)$ lies in $\varphi(A) = B$.
- *restricts to the identity on $(Y \times \{0\}) \cup (B \times I)$* : let (y, t) be a point of this subspace. Since φ^{-1} maps Y to X and B to A , the point $(\varphi^{-1}(y), t)$ lies in $(X \times \{0\}) \cup (A \times I)$, on which r_X is the identity. Therefore $r_X(\varphi^{-1}(y), t) = (\varphi^{-1}(y), t)$, and applying φ to the first component gives $r_Y(y, t) = (y, t)$.

□

Proposition 2.9. *Let $C \subseteq B \subseteq X$ be topological spaces such that both (X, B) and (B, C) have the HEP. Then (X, C) has the HEP.*

Proof. Let Z be a topological space, let $f: X \rightarrow Z$ be a map and let $H: C \times I \rightarrow Z$ be a homotopy with $H(c, 0) = f(c)$ for all $c \in C$. Applying the HEP of (B, C) to the restriction $f|_B$ and to H yields a homotopy $H': B \times I \rightarrow Z$ with

$$H'(b, 0) = f(b) \quad \text{for all } b \in B \quad \text{and} \quad H'|_{C \times I} = H.$$

In particular $H'(b, 0) = f(b)$, so we may apply the HEP of (X, B) to f and H' and obtain a homotopy $H'': X \times I \rightarrow Z$ with $H''(x, 0) = f(x)$ for all $x \in X$ and $H''|_{B \times I} = H'$. The first condition of theorem 2.1 is thus satisfied, and since $C \subseteq B$ we also get $H''|_{C \times I} = H'|_{C \times I} = H$. □

2.2 HEP for relative CW-complexes

To proof the *HEP* for relative CW-complexes, we generalize the pair of spaces in three steps proving the HEP at each stage:

1. Every closed ball relative to its boundary has the HEP.
2. For every relative CW-complex, two consecutive skeleta form a pair with the HEP, that is, (X_{n+1}, X_n) has the HEP for all n .
3. Every relative CW-complex (X, C) has the HEP.

Before turning to the proofs we introduce the definitions and notations we use for CW-complexes.

The following definition of relative CW complexes is a modified version of the definition of CW-complexes given in the bachelor thesis of Hannah Scholz [Sch24]. She formalised the historical definition of CW-complexes, presented by Whitehead in [Whi18], and this formalisation is now available in mathlib as *Classical CW-Complex*. We follow her definition because it provides exactly the structures our formalised proofs rely on: explicit characteristic maps together with the weak topology, rather than an inductive description via attaching cells.

Definition 2.10. We write $D^n := \{x \in \mathbb{R}^n \mid \|x\| \leq 1\}$ n -dimensional closed ball. Its boundary $\partial D^n = S^{n-1} := \{x \in \mathbb{R}^n \mid \|x\| = 1\}$ is the $(n - 1)$ -dimensional sphere.

Definition 2.11. Let X be a Hausdorff space and $C \subset X$ a closed subspace. A *relative CW-complex* on (X, C) consists of a family of indexing sets $(I_n)_{n \in \mathbb{N}}$ together with a family of maps $(Q_i^n : D_i^n \rightarrow X)_{n \geq 0, i \in I_n}$ such that:

1. $Q_i^n|_{\mathring{D}_i^n} : \mathring{D}_i^n \rightarrow Q_i^n(\mathring{D}_i^n)$ is a homeomorphism. We call $e_i^n := Q_i^n(\mathring{D}_i^n)$ an *(open) n -cell* (or a cell of dimension n) and $\bar{e}_i^n := Q_i^n(D_i^n)$ a *closed n -cell*.
2. For all $n, m \in \mathbb{N}$, $i \in I_n$ and $j \in I_m$ with $(n, i) \neq (m, j)$ the cells e_i^n and e_j^m are disjoint.
3. For all $n \in \mathbb{N}$ and $i \in I_n$, the set $Q_i^n(\partial D_i^n)$ is contained in the union of C and a finitely many closed cells of dimension less than n .
4. A subset $S \subseteq X$ is closed if and only if $S \cap C$ is closed and $S \cap \bar{e}_i^n$ is closed for all $n \in \mathbb{N}$ and $i \in I_n$.
5. $C \cup \bigcup_{n \geq 0} \bigcup_{i \in I_n} Q_i^n(D_i^n) = X$.

We call Q_i^n a *characteristic map*, refer to C as the *base* of the complex and call $\partial e_i^n := Q_i^n(\partial D_i^n)$ the *frontier of the n -cell* for any i and n . For $-1 \leq n \leq \infty$ we define the n -*skeleton* of X as

$$X_n := C \cup \bigcup_{m \leq n} \bigcup_{i \in I_m} \bar{e}_i^m.$$

Note that $X_{-1} = C$ and $X_\infty = X$. Moreover, every skeleton is closed in X . This follows from condition (4), since the intersection of X_n with C and with every closed cell is closed. We use this below whenever we apply the converse direction of theorem 2.7.

Definition 2.12. If $C = \emptyset$, we call $(X, \emptyset)$, or simply X , an *(absolute) CW-complex*.

Definition 2.13. A relative CW complex (X, C) is called *finite dimensional* if there exists an $n \in \mathbb{N}$ such that $I_m = \emptyset$ for all $m \geq n$.

2.2.1 HEP for balls

Theorem 2.14. *For every $m \in \mathbb{N}$, the pair $(D^m, \partial D^m)$ has the HEP.*

There are two different attempts to prove this. One argues visually via the retraction criterion, projection the cylinder $D^m \times I$ from a point above it onto $(D^m \times \{0\}) \cup (\partial D^m \times I)$. The other constructs the extended homotopy explicitly. We give the constructive proof, since it was the easier one to formalise in Lean.

Proof. Let Y be a topological space, let $f: D^m \rightarrow Y$ be a map and let $H: \partial D^m \times I \rightarrow Y$ be a homotopy with $H(a, 0) = f(a)$ for all $a \in \partial D^m$. We assemble these into a map $h: D_2^m \rightarrow Y$, where D_2^m denotes the closed ball of radius 2 in $\mathbb{R}^m$, by setting

$$h(x) = \begin{cases} f(x) & \|x\| \leq 1 \\ H\left(\frac{x}{\|x\|}, \|x\| - 1\right), & \|x\| \geq 1 \end{cases}$$

The two cases agree on the sphere $\|x\| = 1$, where the second one gives $H(x, 0) = f(x)$, and both domains are closed in D_2^m , so h is continuous by the pasting lemma. Using h we define

$$H': D^m \times I \rightarrow Y, \quad H'(x, t) = h((1+t)x),$$

which is well defined and continuous, and which satisfies $H'(x, 0) = h(x) = f(x)$ for all $x \in D^m$ as well as $H'(a, t) = H(a, t)$ for all $a \in \partial D^m$ and $t \in I$, since $\|(1+t)a\| = 1+t \geq 1$. $\square$

For $m = 0$ the sphere ∂D^0 is empty and the second case of the definition of h never occurs. In Lean this requires no special treatment, since division by zero is defined to be zero: the term $x/\|x\|$ is total and the case distinction goes through unchanged.

Since the definition of CW-complexes in mathlib uses n -dimensional cubes rather than closed balls as the domain of the characteristic maps, we restate the HEP for cubes relative to their boundary. – ToDo: Question: eigentlich nicht ist der domain nicht der cube, sondern es ist ein partial homeomorphismus mit source cube. Muss man da präzieser sein, oder reicht das, weil es hier nur um die Motivation geht?

Corollary 2.15. *For every $n \in \mathbb{N}$, the pair $([-1, 1]^n, \partial[-1, 1]^n)$ has the HEP.*

Proof. The ball D^n is homeomorphic to the cube $[-1, 1]^n$ via radial scaling, which maps each ray from the origin to itself and takes ∂D^n to $\partial[-1, 1]^n$. This maps the sphere to the boundary of the cube. The claim now follows from theorem 2.14 together with theorem 2.8. $\square$

2.2.2 HEP for consecutive skeletons

In the next step we prove that for every relative CW complex (X, C) and every $m \in \mathbb{N}$, the pair (X_m, X_{m-1}) has the HEP. We do so by applying the retraction criterion, that is, by constructing a retraction

$$r_m^{\text{Skel}}: X_m \times I \rightarrow (X_m \times \{0\}) \cup (X_{m-1} \times I).$$

This map will be obtained from the universal property of the quotient. To use it we need to know that $X_m \times I$ carries the quotient topology, and for this it suffices to know that X_m does, by the following theorem:

Theorem 2.16. *Let Z be a locally compact space and let $g: Y \rightarrow Y'$ be a quotient map. Then*

$$g \times \text{id}_Z: Y \times Z \rightarrow Y' \times Z$$

is a quotient map.

Since the unit interval I is compact, and in particular locally compact, theorem 2.16 applies to $Z := I$ throughout this section. So let us prove that every skeleton indeed is a quotient space: We set $Z_m := C \amalg \coprod_{n \leq m} \coprod_{i \in I_n} D_i^n$ and define

$$P_m: Z_m \rightarrow X_m$$

$$x \mapsto \begin{cases} Q_i^n(x), & x \in D_i^n, \\ x, & x \in C \end{cases}$$

Lemma 2.17. *Let (X, C) be a relative CW-complex. For every $m \in \mathbb{N}$ the map $P_m: Z_m \rightarrow X_m$ is a quotient map.*

Proof. The map P_m is well defined, since the images of the characteristic maps up to dimension m lie in X_m and the base C is contained in every skeleton. It is continuous because its restriction to each summand of the coproduct is, and it is surjective by the definition of X_m . It remains to show that a subset $S \subseteq X_m$ is closed whenever $P_m^{-1}(S)$ is closed. So let $P_m^{-1}(S)$ be closed. By the definition of the coproduct topology this means that

$$P_m^{-1}(S) \cap C = S \cap C \quad \text{and} \quad P_m^{-1}(S) \cap D_i^n = (Q_i^n)^{-1}(S)$$

are closed for all $n \leq m$ and $i \in I_n$. Since X_m is closed in X , it suffices to show that S is closed in X . By condition 4 of theorem 2.11 this amounts to showing, that $S \cap C$ and $S \cap \bar{e}_i^n$ are closed for all $n \in \mathbb{N}$ and $i \in I_n$. – ToDo: argument aus der formalisierung abgleichen

Since X_m is closed in X and the m -skeleton only consists out of the base space and cells up to dimension m . So to prove closedness of a subset of the m -skeleton, it suffices to show, that the intersection with the base space and with all closed cells up to dimension m is closed. This follows from the observation, that $P_m^{-1}(S) \cap D_i^n = S \cap \bar{e}_i^n$ and $P_m^{-1}(S) \cap C = S \cap C$. $\square$

By ?? and theorem 2.16, the map $P_m \times id_I: Z_m \times I \rightarrow X_m \times I$ is a quotient map. We now define

$$f_m: Z_m \times I \rightarrow (X_m \times \{0\}) \cup (X_{m-1} \times I)$$

$$(z, t) \mapsto \begin{cases} (P_m \times id_I) \circ r_i^m(x), & z \in D_i^m \text{ for some } i \in I_m \\ (P_m \times id_I)(z, t), & \text{otherwise} \end{cases}$$

. Here $r_i^m: D_i^m \times I \rightarrow D_i^m \times \{0\} \cup S_i^{m-1} \times I$ is the retraction, obtained from theorem 2.14 and theorem 2.7. Note that the case distinction is unambiguous: the summands of Z_m are disjoint, so z lies in a ball of dimension exactly m for at most one index i . Once we know that f_m is continuous and factors through $P_m \times id_I$, the universal property of the quotient yields a map r_m^{Skel} making the following diagram commute.

$$\begin{array}{ccc} Z_m \times [0, 1] & \xrightarrow{f_m} & X_m \times \{0\} \cup X_{m-1} \times [0, 1] \\ \downarrow P_m \times id_{[0, 1]} & \nearrow \exists r_{\text{Skel}}^m & \\ X_m \times [0, 1] & & \end{array}$$

It is not clear a priori that this map is a retraction. We verify this separately in ??.

Lemma 2.18. *For every $m \in \mathbb{N}$, the map f_m*

(i) *takes values in $(X_m \times \{0\}) \cup (X_{m-1} \times I)$,*

(ii) is continuous, and

(iii) factors through $P_m \times id_I$

Proof. (ii). We treat the two cases of the definition separately. First let $(z, t) \in D_i^m \times I$ for some $i \in I_m$. Since r_i^m is a retraction,

$$r_i^m(z, t) \in (D_i^m \times \{0\}) \cup (S_i^{m-1} \times I).$$

If the second coordinate of $r_i^m(z, t)$ is 0, then $f_m(z, t) \in X_m \times \{0\}$. Otherwise the first coordinate lies in $S_i^{m-1} = \partial D_i^m$, and by condition (3) of theorem 2.11 its image under Q_i^m lies in X_{m-1} , so $f_m(z, t) \in X_{m-1} \times I$.

Now let $(z, t) \in Z_m \times I$ be a point of the second case, so $z \in C$ or $z \in D_i^n$ for some $n < m$. In either case $P_m(z) \in X_{m-1}$, hence $f_m(z, t) \in X_{m-1} \times I$.

(ii). The space $Z_m \times I$ is homeomorphic to $(C \times I) \amalg \coprod_{n \leq m} \coprod_{i \in J_n} (D_i^n \times I)$, via the canonical mapp. Under this homeomorphism, f_m is defined on a coproduct and not a product space anymore. f_m restricts on each summand to a composite of continuous maps, and a map out of a coproduct is continuous if and only if all its restrictions to the summands are. Hence f_m is continuous.

(iii). Let (z, t) and $(z', t') \in Z_m \times [0, 1]$ with $P_m \times id_I(z, t) = P_m \times id_I(z', t')$. Since $P_m \times id_I$ is the identity on the second coordinate, we directly get $t = t'$ and $P_m(z) = P_m(z')$. We must show, that $f_m(z, t) = f_m(z', t)$.

If neither z nor z' lies in a ball of dimension m , both are in the second case of the definition and $f_m(z, t) = (P_m \times id_I)(z, t) = (P_m \times id_I)(z', t) = f_m(z', t)$. The same conclusion holds if z lies in ∂D_i^m for some $i \in I_m$, since r_i^m is the identity there and f_m again agrees with $P_m \times id_I$; likewise for z' . So we may assume that $z \in \mathring{D}_i^m$ for some $i \in I_m$, and it suffices to show $z = z'$.

We have $P_m(z) = Q_i^m(x) \in e_i^m$. Suppose z' did not lie in $\mathring{D}_i^m$. If $z' \in \mathring{D}_j^m$ for some $j \neq i$, then $P_m(z') \in e_j^m$, which is disjoint from e_i^m by condition (2) of theorem 2.11. In all remaining cases — that is, if $z' \in C$, or z' lies in a ball of dimension less than m , or in the boundary of a ball of dimension m — condition (3) gives $P_m(z') \in X_{m-1}$, which is disjoint from e_i^m as well. Both contradict $P_m(z) = P_m(z')$, so $z' \in \mathring{D}_i^m$. Since $Q_i^m|_{\mathring{D}_i^m}$ is a homeomorphism onto e_i^m and hence injective, $P_m(z) = P_m(z')$ forces $z = z'$. $\square$

Theorem 2.19. *Let (X, C) be a relative CW-complex with skeleta $(X_m)_{m \in \mathbb{N}}$, then (X_m, X_{m-1}) has the HEP for every $m \in \mathbb{N}$.*

Proof. By theorem 2.18 and the universal property of the quotient we have a continuous map $r_m^{\text{Skel}}: X_m \times I \rightarrow (X_m \times \{0\}) \cup (X_{m-1} \times I)$ with $r_m^{\text{Skel}} \circ (P_m \times id_I) = f_m$. By theorem 2.7 it remains to show that r_m^{Skel} restricts to the identity on $(X_m \times \{0\}) \cup (X_{m-1} \times I)$. Then the pair then has the HEP, since X_{m-1} is closed in X_m .

Since f_m factors through $P_m \times id_I$, the value $r_m^{\text{Skel}}(x, t)$ does not depend on the choice of a preimage of (x, t) , and we may therefore choose a convenient

one in each case.

Points of $X_{m-1} \times I$. Let $(x, t) \in X_{m-1} \times I$. Then x lies in C or in a closed cell of dimension less than m , so we may choose a preimage (z, t) with $z \in C$ or $z \in D_i^n$ for some $n < m$. Such a z falls under the second case of the definition of f_m . Therefor

$$r_m^{\text{Skel}}(x, t) = r_m^{\text{Skel}}((P_m \times id_I)(z, t)) = f_m(z, t) = (P_m \times id_I)(z, t) = (x, t).$$

Points of $X_m \times \{0\}$. Let $(x, 0) \in X_m \times \{0\}$. If x lies in X_{m-1} , the previous case applies. Otherwise x lies in an open cell of dimension m , so there are $i \in I_m$ and $z \in D_i^m$ with $P_m(z) = x$. Since r_i^m is a retraction and $(z, 0) \in D_i^m \times \{0\}$, we get $r_i^m(z, 0) = (z, 0)$ and therefore

$$r_m^{\text{Skel}}(x, 0) = f_m(z, 0) = (P_m \times id_I)(r_i^m(z, 0)) = (P_m \times id_I)(z, 0) = (x, 0). \quad \square$$

2.2.3 HEP for relative CW complexes

In this final section we prove that every relative CW-complex (X, C) has the HEP. For the finite dimensional complex this follows quickly from what we established so far.

Lemma 2.20. *Let (X, C) be a relative CW-complex. Then (X_n, C) has the HEP for every $n \in \mathbb{N} \cup \{-1\}$.*

Proof. We proceed by induction on n .

IBase case $n = -1$: Note, that $X_{-1} = C$, so the pair in question is $(X_{-1}, C) = (C, C)$, which has the HEP by theorem 2.2.

Inductive step $n \rightarrow n+1$. Assume, we know that (X_n, C) has the HEP. By theorem 2.19 the pair X_{n+1}, X_n has the HEP as well, and since $C \subseteq X_n \subseteq X_{n+1}$, theorem 2.9 yields the HEP for (X_{n+1}, C) . $\square$

Corollary 2.21. *Let (X, C) be a finite dimensional relative CW complex. Then (X, C) has the HEP.*

Proof. Since (X, C) is finite dimensional, there is an $n \in \mathbb{N} \cup \{-1\}$, such that X is equal to X_n . The claim therefore follows from theorem 2.20 applied to X_n . $\square$

This is the point at which our formalisation stops. For the sake of completeness we sketch the proof of the general case, which we did not formalise in Lean.

Lemma 2.22. *Let (X, C) be a relative CW complex. Then (X, C) has the HEP.*

Sketch. We use the retraction criterion, which applies since C is closed in X . By theorem 2.20 and theorem 2.7 there are retractions

$$r_m: X_m \times I \rightarrow (X_m \times \{0\}) \cup (C \times I)$$

for all $m \in \mathbb{N} \cup \{-1\}$. These can be constructed inductively: for $m = -1$ we take the identity, and for $m \geq 0$ we let r_m be the composite

$$X_m \times I \xrightarrow{r_m^{\text{Skel}}} (X_m \times \{0\}) \cup (X_{m-1} \times I) \xrightarrow{id \cup r_{m-1}} (X_m \times \{0\}) \cup (C \times I),$$

where the second map is the identity on $X_m \times \{0\}$ and r_{m-1} on $X_{m-1} \times I$. A straightforward induction shows that these two descriptions agree on the overlap and that $r_m|_{X_{m-1} \times I} = r_{m-1}$ for all m . This compatibility allows us to define

$$r: X \times I \rightarrow (X \times \{0\}) \cup (C \times I), \quad r(x, t) := r_m(x, t) \quad \text{for any } m \text{ with } x \in X_m,$$

which is independent of the choice of m . It restricts to the identity on $(X \times \{0\}) \cup (C \times I)$, since each r_m does. Continuity follows from theorem 2.23 below, applied to $Z := I$: the restriction of r to $X_n \times I$ is r_n and hence continuous for every n . Thus r is a retraction, and the retraction criterion gives the HEP for (X, C) . $\square$

Proposition 2.23. *Let (X, C) be a relative CW-complex, let Z be a locally compact space and let Y be a topological space. A map $f: X \times Z \rightarrow Y$ is continuous if and only if the restriction $f|_{X_n \times Z}$ is continuous for every $n \geq -1$.*

We state theorem 2.23 without proof. It expresses that X carries the colimit topology of its skeleta and that this is preserved under taking the product with a locally compact space, which is the same phenomenon that theorem 2.16 describes for a single quotient map.

3 Lean and mathlib

Having discussed the informal mathematics behind our formalisation, we now give a brief overview of Lean and of the conventions and concepts that are relevant to this project.

Lean is a proof assistant, that is, a tool that checks the individual reasoning steps of a proof written in its formal language. It verifies that assumptions, axioms, functions and theorems are applied only where permitted and that new results follow by valid logical inference. According to its main designers, this verification rests upon a “small trusted kernel based on dependent type theory” (leanTP), a notion we explain in the next section. Lean is one of several proof assistants, alongside Rocq and Isabelle. As Kontorovich describes in *The Shape of Math to Come* [Kon25], it is remarkable how quickly Lean has gained relevance not only in computer science and mathematical logic, but in mainstream mathematical research.

The first version of Lean was created by Leonardo de Moura in 2013, and the first stable release of Lean 4 appeared in September 2023. Since July 2023 the development of the language has been coordinated by the non-profit *Lean Focused Research Organization*, whose stated goal is “enhanced scalability, usability, documentation, and proof automation while guiding Lean toward long-term self-sustainability” [FRO]. Alongside the language, the mathematical library *mathlib* has grown considerably: as of August 2026 it comprises almost 2.500.000 lines of code contributed by over 770 authors [].

3.1 Lean’s Dependent Type Theory

The following description is based on the chapter *Dependent Type Theory* in the digital book *Theorem Proving in Lean 4* [TPIL].

Every expression in Lean has a type, which can be displayed with the command `#check`. The familiar mathematical objects $\mathbb{N}$, $\mathbb{R}$, $\mathbb{C}$ and the Euclidean spaces all have type `Type`, and a function from α to β has type $\alpha \rightarrow \beta$. Types themselves are expressions and therefore have types again: `Type` has type `Type 1`, and in general `Type k` has type `Type (k+1)`. This hierarchy of *universes* is the reason why declarations in later sections carry universe variables such as `u`, introduced automatically when we write `Type*` for a type in an arbitrary universe.

```
#check  $\mathbb{N}$ 
 $\mathbb{N}$  : Type
#check EuclideanSpace  $\mathbb{R}$  (Fin 3)
EuclideanSpace  $\mathbb{R}$  (Fin 3) : Type
#check Type
Type : Type 1
#check 2 = 1
2 = 1 : Prop
```

A distinguished type is `Prop`, the type of propositions. Note that `2 = 1` is a well-formed term of type `Prop`. Propositions need not be true.

What makes this a *dependent* type theory is that a type may depend not only on other types, but on arbitrary terms. We have already seen an instance of this: `Fin : ℕ → Type` assigns to every natural number n the type with n elements, so that `Fin 3` and `Fin 5` are distinct types determined by the values 3 and 5.

3.2 Sets and Subtypes

Type theory allows us not only to define elements of a type, but to build new types from existing ones. The most relevant construction to this project is that of a subtype. Given a type α and a predicate $p : \alpha \rightarrow \text{Prop}$, the subtype `{x :  $\alpha$  // p x}` is the type whose terms are pairs consisting of a term $x : \alpha$ together with a proof of $p x$. Constructing such a term therefore requires supplying both components. In the example below, the zero of the closed unit ball in $\mathbb{R}$. In the example below, the zero of the closed unit ball in $\mathbb{R}$ is given by the real number 0 together with a proof that it lies in the ball. While one would not usually distinguish between this point and the real number zero, in Lean they are terms of different types.

```
#check (<0 , Metric.mem_closedBall_self zero_le_one> : Metric.
closedBall (0 : ℝ) 1)
<0, ...> : { x // x ∈ Metric.closedBall 0 1 }
```

Closely related is the type `Set  $\alpha$` of subset of α , which is defined as $\alpha \rightarrow \text{Prop}$. These two notions are connected by a coercion. Every `S : Set  $\alpha$` can be read as a type `uparrow S`, namely the subtype `{ x :  $\alpha$  // x ∈ S }`, and conversely every term of `↑ S` has an underlying term of α , obtained by `Subtype.val`. Both directions are inserted automatically by Lean where needed. The distinction matters for us because it offers different ways of formalising a pair of spaces (X, A) with $A \subseteq X$, two of which are used in this project. In the first, the larger space is a type `X` and the subspace is a term `A : Set X`. In the second, both are terms of `Set Y` for an ambient type `Y`. In either case a subtype appears as soon as we want to treat a set as a space in its own right. Much of the design of the definitions below is aimed at avoiding subtypes and coercions.

3.3 Structures

A *structure* in Lean bundles several components into a single type. Its declaration lists a number of fields, each with its own type, and a term of the structure is obtained by supplying a term for every field. Structures are how `mathlib` organises mathematical objects: a topological space, for instance, is a structure whose fields consist of a collection of open sets together with

proofs of the axioms it has to satisfy. We use this mechanism in two places, in both cases as a structure whose fields are all propositions. Since proofs of a proposition are indistinguishable in Lean, such a structure carries no data beyond the fact that its conditions hold. It simply gives a name to a conjunction of conditions, so that a single hypothesis can be passed around instead of four separate ones, and each condition can be retrieved by its field name.

3.4 Junk Values

If a formalised statement involves a division by zero, Lean returns the result 0.

```
example : (1 : ℚ) / 0 = 0 := by norm_num
```

This makes division a total function $\mathbb{Q} \rightarrow \mathbb{Q} \rightarrow \mathbb{Q}$, even though certain inputs have no genuine mathematical meaning. As explained in [MLS25], such *junk values* are common practice in Lean. The same convention applies, for instance, to the inversion of square matrices, which is defined for singular matrices as well and returns the zero matrix, or to the derivative of a function at a point where it is not differentiable, which is defined to be zero. At first glance this may seem concerning, since one can state and prove propositions that in classical mathematics would look false or ill-defined. The issue is resolved by adding suitable hypotheses such as $n \neq 0$ or `Differentiable k f` to any statement about the object in question. These ensure that a theorem can only be applied where its conclusion is meaningful. What is gained in return is that definitions and statements become simpler, because no side conditions have to be carried along merely to make an expression well formed. We make frequent use of this convention. Functions that are mathematically defined only on a subset are defined on the entire ambient space. This allows us to extend them without subtype coercions and to apply arguments to them, without proving every time, they lie in the right subset. Junk values also appear in the formalisation of theorem 2.14: the term $x/\|x\|$ occurring in the definition of h is meaningless at $x = 0$, but since division by zero is defined to be zero, no separate treatment of this case is required.

3.5 mathlib

mathlib is a community-driven library of mathematics formalised in Lean. It contains definitions and theorems from a wide range of mathematical fields, and it is continually extended by contributors all over the world. Results are stated as generally as the setting allows, so that they are applicable in as many situations as possible. For this project, we import, among others, the topology packages, which include the formalisation of CW-complexes and some of the basic results about them that our proof relies on.

Names in *mathlib* are chosen to be self-explanatory, in the sense that the name

of a result describes its statement. The theorem that a composition of two continuous functions is continuous, for example, is called `Continuous.comp`. Given `f` and `g` together with proofs `hg: Continuous g` and `hf: Continuous f`, we obtain a proof of `Continuous (g ∘ f)` by writing `Continuous.comp hg hf`, or, in the shorter and more common form using dot notation, `hg.comp hf`. This is possible, because `hg` has type `Continuous g` and the theorem's name begins with `Continuous`, so Lean resolves `hg.comp` to `Continuous.comp` with `hg` inserted as the first explicit argument of that type.

4 Lean formalisation of the Homotopy Extension Property

4.1 CW-complexes in Mathlib

1. NOTIZEN:
2. Zellen im CW complex sind würfel anstatt spheres
3. CW -complexe leben in einem größeren Raum ->

4.2 HEP Definition

Already in the mathematical section, the definition of the *homotopy extension property* was quite long and technical. The same is true for its formalization. Currently I have three versions of the definition in my project, since we decided within the work progress, that different phrasings might be more applicable. I will later on discuss the pros and cons for each of the version. The first definition is the following, called HEP:

```
def agreeOn.{u} {X Y : Type u} (f : X → Y) (H : X × ℝ → Y) (A : Set X) : Prop :=
  ∀ (a : X), a ∈ A → f a = H (a, 0)

structure HomotopyExtension.{u} {X Y : Type u} [TopologicalSpace X]
  [TopologicalSpace Y] (H' : X × ℝ → Y) (f : X → Y)
  (H : X × ℝ → Y) (A : Set X) (rangeH' : Set Y) : Prop where
ContinuousOn : ContinuousOn H' { p : X × ℝ | p.2 ∈ unitInterval }
range : { p : X × ℝ | p.2 ∈ unitInterval }.MapsTo H' rangeH'
agreef : (∀ (x : X), f x = H' (x, 0))
agreeH : (∀ (a : A) (t : unitInterval), H (a,t) = H' (a, t))

def HEP.{u} (X : Type u) [TopologicalSpace X] (A : Set X) : Prop :=
  ∀ (Y : Type u) [TopologicalSpace Y] (rangeH' : Set Y), ∀ (f : X → Y),
  ∀ (H : X × ℝ → Y),
  Continuous f → Set.range f ⊆ rangeH' →
  ContinuousOn H {p : X × ℝ | p.1 ∈ A ∧ p.2 ∈ unitInterval} →
  {p : X × ℝ | p.1 ∈ A ∧ p.2 ∈ unitInterval}.MapsTo H rangeH' →
  agreeOn f H A
  → ∃ H' : X × ℝ → Y, HomotopyExtension H' f H A rangeH'
```

To shorten the definition a little and in order to make it more readable, I first defined `agreeOn`. This tells, that the function `f` and the homotopy `H`, which we want to extend, have the same values on the subset `A` respectively `A × 0`. So it ensures, that the two maps are compatible and it makes sense to ask for an extension. Secondly I constructed the structure called `HomotopyExtension`, which combines all the properties, our desired homotopy should have. Namely, continuity, that its image lies in the right space and

that it agrees with the two functions given in the begin. Making this a structure, instead of a simple definition has the advantage, that it comes with the projection maps for each property.

If we now take a closer look at the definition of the homotopy extension property, it is pretty similar to the mathematical definition ?? of the *HEP*. Still, there is one point, I want to discuss. Defining the homotopies as a map on $X \times \mathbb{R}$ instead of $X \times \text{unitInterval}$ or $A \times \text{unitInterval}$, makes it easier to work with it. For instance consider the definition `agreeOn`. Assume, the homotopy H was defined as a map on $A \times \text{unitInterval}$, then one had to provide a proof of $a \in A$, every time one wants to apply a point $(a : X)$ to the function. Also in the second argument, we can apply any real number to it, without the need to provide a prove that this real number lies between zero and one.

Still, this does not change anything about the defined object. `ContinuousOn H` ' $\{ p : X \times \mathbb{R} \mid p.2 \in \text{unitInterval} \}$ and $\forall (a : A) (t : \text{unitInterval}), H(a, t) = H'(a, t)$ make it irrelevant, what happens outside of the compact interval, but instead put precise focus on what we need for our definition.

Moreover, the set `rangeH'` has no equivalent in the informal definition, but is a result of formalisation. `rangeH'` is a set of the codomain-type Y and as the name indicates, the image of the extended homotopy H' is contained in it. Together with adding this set to the definition, there are two hypotheses, which we need to add. H' cannot have image in `rangeH'`, if not f and H already have image in there.

The win of adding `rangeH'` to the definition is again, that we can avoid working with subtypes. If we want to construct an extended homotopy, which has image in a subtype, we now get a map between types together with the result, that the image lies in the required subset. Nevertheless, in most cases, one wants to choose `rangeH'` to be all of Y . In Lean this can be expressed as `@ Set.univ Y`. The two additional assumptions are then statements, which can be easily proven by `tauto`. Since this extra work is very manageable, I prefer this definition of *HEP* over the following version called *HEPY*, where `rangeH'` is set to be the entire codomain Y . Even tough this one is shorter and closer to the informal version. The `leamm HEP_iff_HEPY` proofs, that those two definitions are equivalent

```
structure HomotopyExtensionY.{u} {X Y : Type u} [TopologicalSpace X]
  [TopologicalSpace Y] (H' : X × ℝ → Y) (f : X → Y)
  (H : X × ℝ → Y) (A : Set X) : Prop where
  ContinuousOn : ContinuousOn H' { p : X × ℝ | p.2 ∈ unitInterval }
  agreef : ∀ (x : X), f x = H' (x, 0)
  agreeH : ∀ (a : A) (t : unitInterval), H (a,t) = H' (a, t)
```

```
def HEPY (X : Type u) [TopologicalSpace X] (A : Set X) : Prop :=
  ∀ (Y : Type u) [TopologicalSpace Y], ∀ (f : X → Y), ∀ (H : X × ℝ
```

```

→ Y), Continuous f →
ContinuousOn H {p : X × ℝ | (p.1 ∈ A) ∧ (p.2 ∈ (unitInterval))}
→ agreeOn f H A
→ ∃ (H' : X × ℝ → Y), HomotopyExtensionY H' f H A

```

The last aspect, I want to elucidate, before progressing to the final third version of the definition is, that I fixed the Type of X and Y to be the same, namely both are `Type u`, for some arbitrary universe u . This was needed, in order to – `ToDo`

Let us now compare this definition with the later version. `HEP'` :

```

open Set.Notation
def HEP' {Y : Type*} [TopologicalSpace Y] (X A : Set Y) : Prop :=
  HEP X (X ↓∩ A)

```

The notation $(X \downarrow \cap A)$ means, consider $(X \cap A)$ as a set of Type `Set X`. Then `HEP'`, is defined in terms of `HEP` for the pair X and $(X \downarrow \cap A)$. The difference to the previous versions of the definition is, that in the definition `HEP`, A is a subset of the type X . That means that the pair of spaces does not live in an ambient space. Whereelse, in the definition `HEP'`, both sets X and A are sets of the same type Y .

We decided to go for the second version of the definition, when we wanted to phrase the *homotopy extension property* for pairs of consecutive skeletons. A skeleton of a relative CW complex in Lean is a subcomplex, whose carrier is a subset of an ambient space. Also it turned out to be the nicer way, to prove lemmas about closed balls and spheres, because I do not have to consider the sphere as a subset of "Type closedBall" anymore, but just as the points with distance one to the origin in a metric space.

Note, that in this definition the functions f and H , which we want to extend are defined on the subtype $\uparrow X$, respectively $\uparrow X \times \mathbb{R}$. Working with those subtypes caused some extra work in the proof of `retraction_criterion_closed'` and also `HEP_trans'`. One could avoid this by modifying the definition `HEP'` in a way, that it is not defined using `HEP`, but constructed explicitly with functions defined on Y , respectively $Y \times \mathbb{R}$. The definition in that case would be longer, since for example continuity of f would change to a `ContinuousOn`-statement. Also, it would be more work to prove equivalence between `HEP` and this newer version, that it was now, where we basically got it for free. Nevertheless I am convinced, it would improve the formalisation of the definition.

To sum up, I value the primed version `HEP'` to be the most useful one to work with. Optimally with the discussed change, to write the definition explicitly with functions defined on the entire space instead of subspaces. Still, I don't think this outdoes the definition `HEP`, in all accounts. Since the statements become longer and also the proof of the retraction criterion would have been longer and more tedious.

4.3 Retraction Criterion

Before turning to the formalisation of the retraction criterion itself, we look at the definition of a retraction. Classically, a retraction r from B onto A is defined via precomposition with the inclusion $A \hookrightarrow B$, as in theorem 2.5. As with the definition of the HEP, we again want to avoid working with subtypes. Instead of defining a map from B to A , we therefore let the retraction be a map from an ambient type X to itself. The structure `RetractionOn` bundles the properties that make such a map a retraction.

```

structure RetractionOn.{u_1} {X : Type u_1} [TopologicalSpace X] (r :
  X → X) (B A : Set X) : Prop where
  subset : A ⊆ B
  continuousOn : ContinuousOn r B
  mapsTo : B.MapsTo r A
  fixesOn : ∀ a ∈ A, r a = a

```

As the name `RetractionOn` suggests, the map r is a self-map of the type X that is not a retraction on all of its domain. It only "retracts" a subset B onto a subset A . The four fields translate the classical conditions as follows. The first property `subset` states that A is contained in B . Note, that omitting this property would define a more general class of self-maps that need not satisfy $r(B) = A$. The field `continuousOn` requires continuity on the relevant subset B only. Since the codomain of a retraction is supposed to be A , the field `mapsTo` requires that r sends every element of B to an element of A . Finally, the classical definition demands that precomposition with the inclusion $A \hookrightarrow B$ gives the identity id_A . In our setting there is no inclusion map, since domain and codomain of r are the same type X . So the condition therefore becomes the requirement that every element of A is fixed by r . This property is called `fixedOn`.

Recall, that the retraction criterion was obtained from two lemmas. The formalisation follows the same structure. Since theorem 2.4 consists of two implications, only one of which requires closedness, we split it into two separate lemmas, `if_HEP_then_extension` and `if_extension_then_HEP`, so that the closedness assumption appears only where it is needed. Their proofs are unspectacular and follow the informal argument of theorem 2.4 closely, as do the formalisations of theorem 2.6 and of the retraction criterion theorem 2.7 itself.

```

lemma retraction_criterion_closed.{u} {X : Type u} [TopologicalSpace
  X] {A : Set X} (hA1 : IsClosed A) :
  HEP X A ↔ ∃ r : X × ℝ → X × ℝ, RetractionOn r {p | p.2 ∈
    unitInterval} {p | p.2 = 0 ∨ p.1 ∈ A ∧ p.2 ∈ unitInterval}

```

Here the retraction is a map on the product $X \times \mathbb{R}$, where X is the larger space of the pair. Although the lemma is phrased in terms of HEP, it can also be applied to HEP' :


```

variable {Y : Type u} [TopologicalSpace Y] (X A : Set Y) (h : HEP' X
A)
#check if_HEP_then_retraction h

```

While this does yield a retraction for the spaces in question, it is not formulated in the spirit of `HEP'`: the retraction still lives on $X \times \mathbb{R}$ for a type X , whereas `HEP'` is stated for subsets of an ambient type. We therefore also formalised the lemma `retraction_criterion_closed'`, in which the retraction is a map from the ambient space $\times \mathbb{R}$ to itself. As before, the Lean file contains the forward implication without the closedness assumption.

```

lemma retraction_criterion_closed'.{u} {Y : Type u} [
  TopologicalSpace Y] (A X : Set Y) (hAX : A  $\subseteq$  X) (hA1 : IsClosed
  A) :
  HEP' X A  $\leftrightarrow$   $\exists$  s : Y  $\times$   $\mathbb{R}$   $\rightarrow$  Y  $\times$   $\mathbb{R}$ , RetractionOn s {p | p.1  $\in$  X  $\wedge$ 
  p.2  $\in$  unitInterval} {p | p.1  $\in$  X  $\wedge$  p.2 = 0  $\vee$  p.1  $\in$  A  $\wedge$  p.2  $\in$ 
  unitInterval}

```

At this point the ambient-space formulation introduces a discrepancy with the informal statement, which went unnoticed during the formalisation. In theorem 2.4 and theorem 2.7 the hypothesis is that A is closed in X , that is, in the subspace topology of X . In `retraction_criterion_closed'`, however, both A and X are subsets of the ambient type Y , and `hA1 : IsClosed A` asserts that A is closed in Y . This is strictly stronger.

In practice this turned out to be convenient rather than restrictive, since all closedness facts we need are available in the ambient space anyway: the skeleta of a relative CW-complex are closed in the ambient type, as is the sphere S^{m-1} in $\mathbb{R}^m$. Ideally one would drop the assumption entirely. Hatcher [Hat02] states the retraction criterion without it, the general case being proved in the appendix following an argument of [Str68]. Formalising that argument would remove the discrepancy, but it is considerably more involved than the proof we give.

We proved the `retraction_criterion_closed'` from `retraction_criterion_closed`. For the forward direction, we obtain a retraction r as in `ToDo (???)`. In order to extend this map $r : \uparrow X \times \mathbb{R} \rightarrow \uparrow X \times \mathbb{R}$ to a map $s : Y \times \mathbb{R} \rightarrow Y \times \mathbb{R}$, we precomposed in the first component with a map `mapYX` : $Y \rightarrow X$, which is the identity on X and sends all other points of Y to an arbitrary junk value. `Subtype.val` sends $(r\ x).1 \in X$, back to the corresponding point of Y . It remains to confirm that this newly defined map is again a retraction, that is, that it is continuous on the relevant set, maps into the required one, and leaves the necessary values unchanged.

For the backward direction we start with from a retraction $s : Y \times \mathbb{R} \rightarrow Y \times \mathbb{R}$ and want to construct $r : \uparrow X \times \mathbb{R} \rightarrow \uparrow X \times \mathbb{R}$, so that `retractioncriterionclosed` gives the claim. One might expect this direction to be the easier one, since no new values have to be constructed and s only needs to be restricted to the desired space. This expectation is disappointed at the point where the image

is required to lie in $X \times I \subseteq Y \times \mathbb{R}$: the map s is a retraction on $\{p \mid p.1 \in X \wedge p.2 \in \text{unitInterval}\}$, so its `RetractionOn.mapsTo` field gives no information about where in $Y \times \mathbb{R}$ a point is sent once its second coordinate lies outside the unit interval. We resolved this by precomposing the real coordinate with `fun t ↦ Set.projIcc (0 : ℝ) 1 zero_le_one t`, where `Set.projIcc` is a continuous map from `mathlib` that retracts a linearly ordered type onto a closed interval in it. After this precomposition the `mapsTo` field of s is applicable to every point we feed into it. This extra work is a consequence of working with types rather than subtypes; it could have been avoided by defining homotopies on $Y \times \text{unitInterval}$ instead of $Y \times \mathbb{R}$.

We should stress that although the two versions of the retraction criterion are phrased in terms of two different definitions of the HEP, this was a matter of style rather than of technical necessity. Also, we experienced both versions valuable in practice. While it might be advisable to settle on the definition `HEP'` alone, both retraction criteria would remain useful.

The value of `retraction_criterion_closed` is illustrated by the proof that two consecutive skeletons satisfy the HEP, where we construct a retraction $r : (\text{skeletonLT } X \ m) \times \mathbb{R} \rightarrow (\text{skeletonLT } X \ m) \times \mathbb{R}$. The key ingredient for the continuity of this map was `Topology.IsQuotientMap.continuous_lift_prod_left`, `mathlib`'s formalisation of theorem 2.16 on products of quotient maps with a locally compact factor. Working with an ambient space here would have destroyed the surjectivity of the quotient map, and the theorem could not have been applied in the form in which `mathlib` provides it.

4.4 Preservation of the HEP

Besides the retraction criterion I formalised two more theorems, which are useful to prove, that a pair of topological spaces has the *HEP*. One is, that the *HEP* is preserved under homeomorphisms. The other is sort of a transitivity property, i.e. if $(A1, A2)$ and $(A2, A3)$ are pairs of topological spaces and both of them satisfy the *HEP*, then also the pair $(A1, A3)$ has the *HEP*.

```
lemma HEP_trans.{u_1} {X : Type u_1} [TopologicalSpace X] (A1 A2 A3 :
  Set X) (h21_sub : A2 ⊆ A1) (h12_hep : HEP' A1 A2) (h32_sub : A3
  ⊆ A2) (h23_hep : HEP' A2 A3) : HEP' A1 A3
```

For the formalisation of the proof, I had the choice between two different attempts. Instead of applying the retraction criterion to all the `HEP` statements, and then using the two retractions from my assumptions to construct the desired retraction, I proved the lemma by checking the definition.

Another positive side effect of this choice was, that I don't have to assume closedness of any `Sets`, which would be required to apply the retraction criterion.

4.5 HEP for discs and cubes

The goal of the file `HEP_ball_cube.lean` is, to prove the homotopy extension property for balls and cubes relative to their boundary. These are not just the first interesting space pairs, we want to prove the *HEP* for, but also the first step in the prove of the *homotopy extension property* for relative CW complexes.

We start with the balls: As in the prove of `??`, I first constructed the auxilliary function `auxfun` combining the information of the given maps `f` and `H`. Then this function is used to explicitly define a desired homotopy `H'`, which we need to check the definition fo *HEP*

```
variable {m : ℕ} {X Y : Type} [TopologicalSpace X] (C : Set Y)
(f : (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) → Y)
(H : (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) × ℝ →
  Y)

def aux_fun : EuclideanSpace ℝ (Fin m) → Y := fun p =>
  if hp : norm p ≤ 1 then f ⟨p, by simp[hp]⟩
  else H (⟨((norm p)-1 : ℝ) · p, inv_norm_smul_mem_unitClosedBall p⟩,
    norm p - 1)

def H' : (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) × ℝ
  → Y :=
  fun p => (aux_fun f H) ((1 + p.2) · p.1)
```

I split up the `ContinuousOn` proof of `H'` in three steps. I started with a lemma, that states `aux_fun` is continuous on the ring $\{x \mid 1 \leq \|x\| \leq 2\} \subset \mathbb{R}^m$ and used this to proof continuity of `aux_fun` on the ball with radius two. The ball with radius two can be written as the union of a ball with radius one and the ring. Continuity on both closed subsets yields continuity on their union. The continuity of `H'` follows from composing two `ContinuousOn` functions. The proofs for all desired properties of `H'` are not hard. Solving, rewirting and confirming inequalities make up big parts of them. All those results combined proof the lemma `HEP_disc_boundary`. The primed version `HEP_disc_boundary'`, is definitionally equal.

```
lemma HEP_disc_boundary : ∀ (m : ℕ),
  HEP (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) (Metric.
    sphere ⟨0, by simp⟩ 1) := by ...

lemma HEP_disc_boundary' : ∀ (m : ℕ),
  HEP' (Metric.closedBall (0 : EuclideanSpace ℝ (Fin m)) 1) (Metric.
    sphere 0 1) := HEP_disc_boundary
```

The `Metric.sphere` in Lean has type `Set α`, where α is the Type of the center (in our case the 0 in an n -dimensional euclidian space). By the way *HEP* is defined, in `HEP_disc_boundary` the sphere is a subset of the closed ball and we have to add the `by simp` proof, to construct the element of the

subtype. On the other hand, for HEP' both spaces live in the same ambient space. We do not have to add a proof Lean interprets the zero as the origin of $\mathbb{R}^m$. Here we see an advantage of the HEP' definition in action, since we can reduce the use of subtypes.

In the formalisation of (relative) CW-complexes, each cell is the image of a closed ball in $(\text{Fin } m \rightarrow \mathbb{R})$ for some m under a characteristic map. Same as `EuclideanSpace  $\mathbb{R}$  (Fin m)`, also `Fin m  $\rightarrow$   $\mathbb{R}$` is a way to formalize a m -dimensional real vectorspace. The relevant difference between them is, that the `EuclidianSpace` comes with the Euclidian norm, whereas on the other version, the default norm is the maximum norm. So the closed ball, which is a preimage of a closed cell in a CW complex, is something we usually would call a cube, not a ball. To prove the HEP for those cubes, we want to apply the fact, that a m -dimensional ball is homeomorphic to a cube of the same dimension. Instead of explicitly construction this homeomorphism, I wanted to obtain it from the following mathlib lemma. It states, that "if s is a convex bounded set with a nonempty interior in a real normed space, then there is a homeomorphism of the ambient space to itself that sends the interior of s to the unit open ball and the closure of s to the unit closed ball."

```
exists_homeomorph_image_interior_closure_frontier_eq_unitBall.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [NormedSpace  $\mathbb{R}$  E] {s : Set
    E} (hc : Convex  $\mathbb{R}$  s) (hne : (interior s).Nonempty) (hb :
      Bornology.IsBounded s) :
   $\exists$  h,  $\uparrow$ h '' interior s = ball 0 1  $\wedge$   $\uparrow$ h '' closure s = closedBall 0
    1  $\wedge$   $\uparrow$ h '' frontier s = sphere 0 1
```

It is quite obvious, that the `Metric.closedBall`, for which we proved the HEP already, is closed, convex and nonempty and therefore satisfies all assumptions. Still we don't want to apply the above lemma directly on this. The statement would be trivial and we just obtained the identity map on the m -dimensional Euclidian space. Instead we first map this closed ball to `Fin m  $\rightarrow$   $\mathbb{R}$` via the homeomorphism

```
def homeo_euclid_max : EuclideanSpace  $\mathbb{R}$  (Fin m)  $\simeq_t$  (Fin m  $\rightarrow$   $\mathbb{R}$ )
  where
  toFun := fun p  $\mapsto$  p
  ...
```

`homeo_euclid_max` maps each point to itself while changing the norm of the space. So the image of the `Metric.closedBall` is still the same round, convex set, even though it is not the same as a closed ball in the codomain-space. Applying the lemma to the image of the `Metric.closedBall` yields a homeomorphism from the `Fin m  $\rightarrow$   $\mathbb{R}$` to itself:

```
def G : (Fin m  $\rightarrow$   $\mathbb{R}$ )  $\simeq_t$  (Fin m  $\rightarrow$   $\mathbb{R}$ ) := (
  exists_homeomorph_image_interior_closure_frontier_eq_unitBall
  convex_euclid_to_max_ball nonempty_euclid_to_max_ball
  bounded_euclid_to_max_ball).choose
```

The composition of those two homeomorphisms is especially a partial homeomorphism $\text{EuclideanSpace } \mathbb{R} \text{ (Fin } m \text{)}$ to $\text{Fin } m \rightarrow \mathbb{R}$ with source the closed ball and target the closed cube. Now applying the lemma `PartialHomeomorph_HEP'` proves

```
lemma HEP_cube_boundary' (m : ℕ) : HEP' (closedBall (0 : (Fin m → ℝ)) 1) (sphere 0 1) := by ...

lemma HEP_cube_boundary (m : ℕ) : HEP (closedBall (0 : (Fin m → ℝ)) 1) (sphere ⟨0, by simp⟩ 1) := by
  exact HEP_cube_boundary' m
```

Before concluding this section, I want to point out, that here the definition of `HEP'` matches very good the structure of the partial homeomorphis. When I first proved this result - before defining `HEP'` - for `HEP`, I had to show that the `Subtze.val` of a sphere in the closed ball is the same as the sphere in the ambient space and the same for the closed ball itself. With `HEP'` we avoid this subtype coercions and the proof is more straight forward. From this result the statement for `HEP` follows directly.

4.6 HEP for consecutive skeletons

Now we want to continue with the second step in the proof of the *HEP* for relative CW complexes, namely that the pair of two consecutive skeletons satisfies the *HEP*. The formalisation of this result builds on the following mathlib lemma. It replaces the theorem, which states, that the product of a quotient map with the identity on a locally compact space, is again a quotient map.

```
theorem Topology.IsQuotientMap.continuous_lift_prod_left {X₀ : Type u_1} {X : Type u_2} {Y : Type u_3} {Z : Type u_4} [
  TopologicalSpace X₀] [TopologicalSpace X] [TopologicalSpace Y] [
  TopologicalSpace Z] [LocallyCompactSpace Y] {f : X₀ → X} (hf :
  IsQuotientMap f) {g : X × Y → Z} (hg : Continuous fun (p : X₀ ×
  Y) => g (f p.1, p.2)) :
  Continuous g
```

We get from this theorem, that the map `g`, which is defined on the product of a quotient space with a locally compact space, is under a bunch of assumptions continuous. We want this `g` to be the retraction map $m\text{-skeleton} \times [0, 1] \rightarrow (m-1)\text{-skeleton} \times [0, 1]$, which yields the *HEP* for the consecutive skeletons. Let us take a closer look at which functions we need to define and which results we have to proof, in order to get there.

4.6.1 Skeleton is a quotient space

First of all we need a quotient map, called `f` in the theorem above, to the $m\text{-skeleton}$. For this we use the fact, that we can "unglue" the skeleton into

the basespace and the disjoint union of closed balls (cubes to be precise) in all dimensions up to m . The identity map on the base space assembles together with the characteristic maps on each closed ball to the map `SkeletonProjection`.

```

abbrev ungluedSkel {Y : Type*} [hY : TopologicalSpace Y] [T2Space Y]
  (X : Set Y) (C : Set Y)
  [RelCWComplex X C] (m : ℕ) :=
  C ⊕ (Σ (n : Fin m) (c : cell X n), (closedBall (0 : Fin n → ℝ)
    1))

def SkeletonProjection {Y : Type*} [TopologicalSpace Y] [T2Space Y]
  (X : Set Y) {C : Set Y} [RelCWComplex X C] (m : ℕ) : ungluedSkel
  X C m → skeletonLT X m := Sum.elim (fun c ↦ ⟨c, by ...⟩) (fun
  ⟨n, i, x⟩ => ⟨map n i x, by ...⟩)

```

To prove, that this indeed defines a quotient map, we have to check, that it is surjective, continuous and induces the topology on the skeleton. The skeleton is a subcomplex of the (relative) CW complex and the topology is defined by characterising the closed sets. A set of the subcomplex is closed if and only if intersected with the base space and any closed cell it is closed in those spaces.

The surjectivity proof is straight forward: A point in the skeleton lies either in the base space or an open cell. In each case there exists a preimage in the base space respectively an open ball. Also continuity is not much work, since `SkeletonProjection` is defined in terms of continuous functions on a disjoint union. Proving that the topology on the skeleton is induced, required some more effort.

We start with a set S , which is of Type `Set` (`skeletonLT X ↑m`), and the assumption, that its preimage under the `SkeletonProjection` is closed in the unglued skeleton. The goal is to show, that the set S is closed in the skeleton. In mathlib the results about closed sets are always phrased for subsets of the ambient space together with the assumption, that the set is contained in the CW complex or the subcomplex. To get into this setting, we need the following lines in the proof:

```

set S' : Set Y := Subtype.val '' S with hS'
have hS'sub : S' ⊆ (skeletonLT X m) := by
  rintro _ ⟨a, _, rfl⟩;
  exact a.2

```

Now we can apply the lemma `Topology.RelCWComplex.isClosed_of_isClosed_inter_openCell_or_isClosed`, which states that S' is closed in the relative CW complex, if the intersection with the base space is closed and for every cell either $S' \cap \text{openCell } n \ j$ or $S' \cap \text{closedCell } n \ j$ is closed. I proved the case with the base space in the lemma `hClosedBase` and the case with cells of dimension less than m in the lemma `hClosedLT`. For higher dimensions the intersection of S' with the open cell is empty and hence closed.

4.6.2 construction of the retraction

The next preparation we take a look at, is the construction the map r . r is supposed to be the retraction for consecutive skeletons. In this section I want to take a closer look, at the definition of this map r , as well as at the related proofs related to this construction.

Note, that the `SumMap`, was called f in the mathematical section.

Same as in the mathematical proof, also in its formalisation, we don't explicitly define the retraction map r on the m -skeleton $\times [0, 1]$, but instead define, what it should do, on the `ungluedSkel` $\times [0, 1]$. This map, will be called `SumMap` and is used in the following way, to define the r .

```
def r (m : ℕ) : skeletonLT X (m + 1) × unitInterval → skeletonLT X
    (m + 1) × ℝ :=
  Function.extend (fun p ↦ ((SkeletonProjection X (m + 1) p.1), p
    .2)) (sumMap m) (Prod.map id Subtype.val)
```

`Function.extend` works in a way, that whenever a point $x \in \text{skeletonLT } X (m + 1)$ has a preimage $p \in \text{ungluedSkel} \times [0, 1]$ under the map $(\text{fun } p \mapsto ((\text{SkeletonProjection } X (m + 1) p.1), p.2))$, then $r \ x = \text{sumMap } m \ p$. Otherwise $r \ x = (\text{Prod.map id Subtype.val}) \ x$. It is not captured in this definition, but we know, that the map `SkeletonProjection` $\times$ `id` is surjective, so the second case of the `Function.extend` could be filled with any junk function, since it will be never applied. One could read the definition of r just as, take any preimage under `SkeletonProjection` and then apply `sumMap` to it.

It might be irritating, that r , which is supposed to be a retraction, is formalised with the second factor of the domain only being the unit interval. Usually we would choose this to be the real numbers, since the retractions are defined with $\mathbb{R}$ instead of the unit interval. But observe, that the theorem `Topology.IsQuotientMap.continuous_lift_prod_left` yields `Continuous` and not `ContinuousOn`. Therefore we should not make the domain any bigger, than what is actually mathematically relevant. In the codomain we do not have this type of problems and can just choose Z to be the skeleton times the entire real line.

```
def cubesretract {m : ℕ} (n : Fin (m + 1)) :
  (closedBall (0 : Fin (n : ℕ) → ℝ) 1) × unitInterval →
  (closedBall (0 : Fin (n : ℕ) → ℝ) 1) × ℝ := fun p ↦
  if n = m then (r_cube n (p.1, Subtype.val p.2))
  else (p.1, Subtype.val p.2)

def sumMap (m : ℕ) :
  (ungluedSkel X C (m + 1)) × unitInterval → skeletonLT X (m+1)
  × ℝ := fun p ↦
```

```

p.1.elim
  (fun c => ((c.val, memBase_memSkeletonLT X c.prop (m + 1)), p
.2))
  (fun ⟨n, i, x⟩ => ((map n i (cubesretract n (x,p.2)).1,
cubesretract_mapsto_weak n i (x,p.2)),
  (cubesretract n (x,p.2)).2))

```

SumMap is defined using Sum.elim, which is a "case analysis for sums that applies the appropriate function [...] after checking which constructor is present", i.e. whether the first coordinate of the applied point p lies in the base space C or in a closed ball. In the first case, we want to do nothing, but apply the SkeletonProjection on this first coordinate. On the base space, this was just the "identity". So we send the a point $c : C$, to itself, but as an element of the m -skeleton. Since the prove, that a point from the base space lies any desired skeleton is used more often, I made this an own lemma called memBase_memSkeletonLT.

For the second case, that the first coordinate of the applied point p comes from a closed ball, we need the map cubesretract. cubesretract is the retraction map for the *HEP* of cubes relative their boundary, if the we are in the balls of top dimension and just a simple subtype coercion in all other dimension. For cubesretract we proved small lemmas about continuuity, where the function is fixed and in which set the image of the function lies.

```

lemma cubesretract_continuous {m : ℕ} (n : Fin (m + 1)) : Continuous
  (cubesretract n) := by ...

```

```

lemma cubesretract_fixedOn {m : ℕ} {n : Fin (m + 1)} (x : (
closedBall (0 : Fin n → ℝ) 1) × unitInterval) :
  (n < m → cubesretract n x = (x.1, x.2.1)) ∧
  (n = m → x ∈ {z | z.2 = 0 ∨ z.1 ∈ sphere ⟨0, by simp⟩ 1 } →
cubesretract n x = (x.1, x.2.1)) := by ...

```

```

lemma cubesretract_mapsto_lt {m : ℕ} (n : Fin (m + 1)) (i : cell X n
) (p : (closedBall 0 1) × ↑unitInterval) (hn : n < m) :
  (map n i) (cubesretract n p).1 ∈ skeletonLT X m ∧ (cubesretract
n p).2 ∈ unitInterval
:= by ...

```

```

lemma cubesretract_mapsto_top {m : ℕ} (n : Fin (m + 1)) (i : cell X
n) (p : (closedBall 0 1) × ↑unitInterval) (hn : n = m) :
  (cubesretract n p).2 = 0 ∨ (map n i) (cubesretract n p).1 ∈
skeletonLT X m ∧ (cubesretract n p).2 ∈ unitInterval := by ...

```

So in this described second case, we first apply the map cubesretract to the point and then postcompose again with what SkeletonProjection $\times$ id $_{\mathbb{R}}$ would do to a such point. Since the first coordinate, also after applying cubesretract lies in a closed ball, SkeletonProjection is the same as applying the charakteristic map of that closed ball. We have to include a poof, that the

image of this characteristic map actually lies in the m -skeleton. Also this proof in an own lemma, called `cubesretract_mapsto_weak`, which is derived from the stronger statements above.

Taking a closer look at the definition of `SumMap` it is, in the case, that the point p comes from the base space or a closed ball of dimension less then m , just the same as applying the `SkeletonProjection × idℝ`. At least when ignoring the Subtype issues. It would have been inconvenient to write this application in the definition of `SumMap`, so instead I proofed the two lemma, which show exactly this equality.

```
lemma sumMap_eq_SkelProj_C (m : ℕ) (x : C) (t : unitInterval) :
  sumMap m (Sum.inl x, t) =
  Prod.map id (Subtype.val) (SkeletonProjection X (m + 1) (Sum.inl
    x), t) := by ...
```

```
lemma sumMap_eq_SkelProj_sphereLT {m : ℕ}
  (n : Fin (m + 1)) (i : cell X n) (x' : closedBall (0 : Fin n →
    ℝ) 1) (t : unitInterval) (hx : n < m ∨ n = m ∧ x' ∈ sphere ⟨0,
  by simp⟩ 1) :
  sumMap m (Sum.inr ⟨n, ⟨i, x'⟩⟩, t) = Prod.map (SkeletonProjection
    X (m + 1)) Subtype.val (Sum.inr ⟨n, ⟨i, x'⟩⟩, t) := by ...
```

4.6.3 Continuity of the retraction

Now, that we defined all relevant maps, we can focus on how exactly we proof the continuity of r . As said earlier, we will get the continuity from the theorem `Topology.IsQuotientMap.continuous_lift_prod_left`. Let's have a look at what is left to proof:

```
lemma continuousRetract (m : ℕ) : Continuous (r (X := X) m) := by
  refine (SkeletonProjectionIsQuotientMap (m + 1) X).
  continuous_lift_prod_left ?_
  ...
```

After this first line of proof the current goal changes to

```
Continuous fun (p : (ungluedSkel X C m) × [0,1]) => r (
  SkeletonProjection p.1, p.2)
```

Showing this boils down to two major steps. The first one is, that `SumMap` factors through `(fun p ↦ ((SkeletonProjection X (m + 1) p.1), p.2))`. From this we can easily show that the continuity goal is equivalent to showing, that `SumMap` is continuous. Continuity of `SumMap` then is the second step. We start with

```
lemma factors (m : ℕ) : (sumMap m).FactorsThrough (fun p ↦ ((
  SkeletonProjection X (m + 1) p.1), p.2)) := by
  intro x y hxy
  ...
```

x and y are two points in $\text{ungluedSkel} \times [0,1]$. Their second coordinate is the same and they have the same image under $\text{SkeletonProjection}$. Now we have to show, that $\text{SumMap } m \ x = \text{SumMap } m \ y$. With [lemma `sumMap\_eq\_SkelProj\_C`](#) and [lemma `sumMap\_eq\_SkelProj\_sphereLT`](#) (printed above) we already have usefull tools to prove this goal. A tricky part, that requird qute some work is:

```
lemma SkeletonProj_inj_top {Y : Type*} [TopologicalSpace Y] [T2Space
  Y] (X : Set Y) {C : Set Y} [RelCWComplex X C] {m : ℕ} {n : Fin
  (m + 1)} (hnm : ↑n = m) (i : cell X n) (x' : closedBall (0 : Fin
  n → ℝ) 1) (x_ball : x'.1 ∈ ball (0 : Fin n → ℝ) 1) (y :
  ungluedSkel X C (m + 1)) (hskel : SkeletonProjection X (m + 1) (
  Sum.inr ⟨n, ⟨i, x'⟩⟩) = SkeletonProjection X (m + 1) y) :
  y = Sum.inr ⟨n, i, x'⟩ := by
```

The lemma `SkeletonProj_inj_top` tells us, that whenever given one point x in an specific open ball of top dimension, and any other point y in ungluedSkel . If they have the same image under $\text{SkeletonProjection}$, then y is the exact same point as x .

The prove of `factors` is long, because it is split into many different case distinctions. Begining with whether x lies in the base space or an closed ball. Then in each case I considered the same two options for y seperately. In the case, that x is in an closed ball, we look at the cases that it is in the top dimesnioal open cell, or not. Most of those cases themselves are quite short. In the mathematical explination it was sufficient to just look at the case, that a point is in the top dimensional cell, or not. So why do I split up the situation so much more in the formalisation? – `ToDo`: weil ich nicht einfach sagen kann: x in C sondern es ein c gibt,... und dann ist es einfacher, wie ich es gemacht habe

For continuity of `SumMap` we precompose the map with the homeomorphism

```
def homeoProd (m : ℕ) :
  (C ⊕ (Σ (n : Fin (m + 1)) ( _ : cell X n), closedBall (0 : Fin n
  → ℝ) 1)) × unitInterval ≃t
  (C × unitInterval) ⊕ (Σ (n : Fin (m + 1)) ( _ : cell X n),
  (closedBall (0 : Fin n → ℝ) 1) × unitInterval) := ...
```

This composite has as a domain not a product anymore, but instead a union. And on each summand the continuity goal is proven by just combining resluts proven earlier such as `cubesretract_continuous` and continuity of the characteristic maps. Since continuity is preserved under precomposition with a homeomorphism, this yields, that `SumMap` and finally also r is continuous.

4.6.4 Properties of the reatraction

In this section I want to conclude the proof of the *HEP* for consecutive skeletons. For the proof via the retraction criterion we have to make a small adjustment to the map r , which we worked with before. I mentioned in the

beginning, that despite the fact, the desired retraction should have $(\text{skeletonLT } X (m + 1)) \times \mathbb{R}$ as a domain, r is just defined on the unit interval in the second component. I experienced it to be the easiest way to extend r by precomposing the second coordinate with the map `ProjIcc`, which was defined in the very first file. Therefore the final definition looks like below.

```
def retr_skeleton (m : ℕ) : (skeletonLT X (m + 1)) × ℝ → (
  skeletonLT X (m + 1)) × ℝ := fun p ↦
  (r m) (p.1, (ProjIcc p.2, proj_mem p.2 ))
```

Now it is only left to show, that `retr_skeleton` is continuous, maps to the right space and is fixed, where required. The continuity follows directly from all the work we did before. The claims about `mapsTo` and `fixedOn` are first proven for `SumMap` and then transferred to `retr_skel`. All together this results in the following lemma.

```
lemma HEP'_skeleton (m : ℕ) : HEP' (skeletonLT X (m + 1)).carrier (
  skeletonLT X m).carrier := by
  apply (retraction_criterion_closed (IsClosed.preimage_val (
    skeletonLT X ↑m).closed')) .2
  use retr_skeleton m
  ...
```

ToDo: man könnte hier noch auf `subst` eingehen, weil das ein game changer war.

4.7 HEP for finite dimensional relative CW complexes

In this last part about my formalisation we will get to the result, that finite dimensional relative CW complexes satisfy the *HEP*. In the chapter about the mathematics it was already visible, that the finite dimensional case follows straight from what we showed earlier by induction, where else the general case requires much more work. Due to the limited time for this thesis I had to stop at this point.

The induction looks in Lean as follows:

```
lemma HEP_skelLT_base (m : ℕ) : HEP' (skeletonLT X m) C := by
  induction m with
  | zero =>
    rw [ENat.coe_zero, skeletonLT_zero_eq_base, HEP', Subtype.
      coe_preimage_self]
    exact HEP_self
  | succ n n_ih =>
    apply HEP_trans (skeletonLT X (n + 1)).carrier (skeletonLT X n)
      C (skeletonLT_mono
        le_self_add) ?_ (Subcomplex.base_subset (skeletonLT X ↑n))
      n_ih
    exact HEP'_skeleton n

lemma HEP_skel_base (m : ℕ) : HEP' (skeleton X m) C := by
```

```
rw [skeleton.congr_simp X m m rfl]
exact HEP_skelLT_base X (m + 1)
```

The induction start, is exactly the lemma `HEP_self`, which we proved in the very begining just after defining the *HEP*. For the induction step we combine via transitivity the induction hypothesis and the *HEP* for consecutive skeletons.

During my project I used to work the whole time, as recommenden by Hannah Scholz with `SkeletonLT`, instead of `Skeleton`. And only defived the statement for `Skeleton` from this. `SkeletonLT` is especially usefull for inductions, as demonstrated above, because it also includes the case of the (-1) -skeleton, which would otherwise be left out.

ToDo Ausblick: In the prvious secion, where we proved the *HEP* for consecutive sekeltions, I used the lemma `retractioncriterionclosed`, where the retraction map is defined on the Type

ToDo SkelLT und skel erklären

5 Methodes and Usage of AI

6 Summary und Ausblick

References

- [] URL: <https://www.math.uni-bielefeld.de/~fw/> (visited on 08/14/2026).
- [] *Mathlib Statistics*. URL: https://leanprover-community.github.io/mathlib_stats.html (visited on 07/02/2026).
- [FRO] Lean FRO. *FRO - About us*. URL: <https://lean-lang.org/fro/about/> (visited on 07/02/2026).
- [Hat02] Allen Hatcher. *Algebraic topology*. Cambridge: Cambridge University Press, 2002, pp. xii+544. ISBN: 0-521-79540-0.
- [Kon25] Alex Kontorovich. *The Shape of Math To Come*. 2025. arXiv: 2510.15924 [math.HO]. URL: <https://arxiv.org/abs/2510.15924>.
- [MLS25] Alex Meiburg, Leonardo A. Lessa, and Rodolfo R. Soldati. *A Formalization of the Generalized Quantum Stein’s Lemma in Lean*. 2025. arXiv: 2510.08672 [quant-ph]. URL: <https://arxiv.org/abs/2510.08672>.
- [Sch24] Hannah Scholz. “Formalisation of CW-complexes”. Bachelor’s Thesis Mathematics. Rheinischen Friedrich-Wilhelms-Universität Bonn, 2024. URL: <https://florisvandoorn.com/theses/HannahScholz.pdf>.
- [Str68] Arne Strøm. “Note on Cofibrations II”. In: *Mathematica Scandinavica* 22 (1968), pp. 130–142.
- [Whi18] J. H. C. Whitehead. “Combinatorial homotopy. I”. In: *Bulletin (new series) of the American Mathematical Society* 55.3 (2018), pp. 213–245. ISSN: 0273-0979.